

FICHA

Nathaly Campos - C.I. 32.460.314

17 de mayo de 2026

Guía de Referencia Didáctica: Ensamblador MIPS

1. Operaciones Aritméticas

- **add** (Suma)
 - **Ensamblador:** add \$s1, \$s2, \$s3
 - **Lógica:** $\$s1 = \$s2 + \$s3$
 - **Explicación:** Suma los valores de dos registros (\$s2 y \$s3) y almacena el resultado en un tercer registro (\$s1).
- **sub** (Resta)
 - **Ensamblador:** sub \$s1, \$s2, \$s3
 - **Lógica:** $\$s1 = \$s2 - \$s3$
 - **Explicación:** Resta el valor de \$s3 al de \$s2 y guarda el resultado en \$s1.
- **addi** (Suma inmediata)
 - **Ensamblador:** addi \$s1, \$s2, 100
 - **Lógica:** $\$s1 = \$s2 + 100$
 - **Explicación:** Suma una constante fija (en este caso 100) directamente al valor de un registro.

2. Transferencia de Datos (Memoria y Arreglos)

- **lw** (Load Word - Cargar palabra)
 - **Ensamblador:** lw \$s1, 100(\$s2)
 - **Lógica:** $\$s1 = \text{Memoria}[\$s2 + 100]$
 - **Explicación:** Lee un bloque de 32 bits desde una posición de memoria (como al acceder al índice de un arreglo estructurado) y lo trae al registro \$s1.
- **sw** (Store Word - Guardar palabra)
 - **Ensamblador:** sw \$s1, 100(\$s2)
 - **Lógica:** $\text{Memoria}[\$s2 + 100] = \$s1$
 - **Explicación:** Toma el dato que ya procesaste en el registro \$s1 y lo guarda de vuelta en una posición de memoria.

- **lh / sh** (Load/Store Half) y **lb / sb** (Load/Store Byte)
 - **Ensamblador:** `lh $s1, 100($s2)` — `sb $s1, 100($s2)`
 - **Explicación:** Hacen exactamente lo mismo que `lw` y `sw`, pero transfiriendo bloques más pequeños: media palabra (16 bits) o un solo byte (8 bits). Útiles para procesar caracteres o números pequeños.
- **lui** (Load Upper Immediate)
 - **Ensamblador:** `lui $s1, 100`
 - **Lógica:** $\$s1 = 100 * 2^{16}$
 - **Explicación:** Carga un número constante directamente en la mitad superior (los 16 bits de más a la izquierda) del registro.

3. Operaciones Lógicas

- **and / or / nor** (Lógica bit a bit)
 - **Ensamblador:** `and $s1, $s2, $s3`
 - **Lógica:** $\$s1 = \$s2 \& \$s3$
 - **Explicación:** Compara los bits de dos registros uno por uno aplicando la compuerta lógica correspondiente (AND, OR, NOR).
- **andi / ori** (Lógica con constante)
 - **Ensamblador:** `andi $s1, $s2, 100`
 - **Explicación:** Aplica la compuerta lógica comparando un registro contra un valor constante que tú defines.
- **sll / srl** (Shift Left/Right Logical - Desplazamiento)
 - **Ensamblador:** `sll $s1, $s2, 10`
 - **Lógica:** $\$s1 = \$s2 \ll 10$
 - **Explicación:** Mueve los bits hacia la izquierda (`sll`) o derecha (`srl`). Es el método más rápido en hardware para multiplicar o dividir por potencias de 2.

4. Saltos Condicionales (Condicionales lógicos)

- **beq / bne** (Branch on Equal / Not Equal)
 - **Ensamblador:** `beq $s1, $s2, L` — `bne $s1, $s2, L`
 - **Lógica:** `if ($s1 == $s2) ir a L` — `if ($s1 != $s2) ir a L`
 - **Explicación:** Compara dos registros. Si son iguales (`beq`) o si son diferentes (`bne`), el programa salta a la línea marcada con la etiqueta "L". Son la base para construir estructuras condicionales simples.
- **slt / slti** (Set on Less Than)
 - **Ensamblador:** `slt $s1, $s2, $s3` — `slti $s1, $s2, 100`
 - **Lógica:** `if ($s2 < $s3) $s1 = 1; else $s1 = 0`

- **Explicación:** Evalúa si un registro es menor que otro (o que una constante). Si la afirmación es cierta, guarda un 1 en **\$s1**. Si es falsa, guarda un 0.

5. Saltos Incondicionales (Ciclos y Acciones Nominadas)

- **j** (Jump - Salto)
 - **Ensamblador:** `j 2500`
 - **Lógica:** ir a la dirección 10000
 - **Explicación:** Obliga al programa a saltar directamente a una parte del código sin evaluar nada. Esencial para reiniciar ciclos y mantener un bucle rodando.
- **jal** (Jump and Link - Salto y enlace)
 - **Ensamblador:** `jal 2500`
 - **Explicación:** Salta a una subrutina (acción nominada), pero antes de irse, guarda la dirección actual en el registro **\$ra** para saber el camino de vuelta.
- **jr** (Jump Register - Retorno)
 - **Ensamblador:** `jr $ra`
 - **Explicación:** Lee la dirección que está guardada en **\$ra** y salta de vuelta a ese punto. Se utiliza siempre al final de una acción nominada para retornar al flujo principal.